Finally, we need to modify the second migration to indicate that `cart_id` can be null in records. This is done by modifying the existing `add_reference` line to say `null: true` and adding a new `change_column` line to enable nulls in the `cart_id` column.

rails80/depot_o/db/migrate/20250420000009_add_order_to_line_item.rb

```
    class AddOrderToLineItem < ActiveRecord::Migration[8.0]
      def change
»       add_reference :line_items, :order, null: true, foreign_key: true
»       change_column :line_items, :cart_id, :integer, null: true
      end
    end
```

Now that we've created the migrations, we can apply them:

```
depot> bin/rails db:migrate
== 20250420000007 CreateOrders: migrating ==============================
-- create_table(:orders)
-> 0.0007s
== 20250420000007 CreateOrders: migrated (0.0022s) =====================

== 20250420000008 AddOrderToLineItem: migrating ========================
-- add_reference(:line_items, :order, {:null=>true, :foreign_key=>true})
   -> 0.0058s
-- change_column(:line_items, :cart_id, :integer, {:null=>true})
   -> 0.0046s
== 20250420000008 AddOrderToLineItem: migrated (0.0246s) ===============
```

Because the database didn't have entries for these two new migrations in the `schema_migrations` table, the `db:migrate` task applied both migrations to the database. We could, of course, have applied them separately by running the migration task after creating the individual migrations.

Creating the Order Capture Form

Now that we have our tables and our models as we need them, we can start the checkout process. First, we need to add a Checkout button to the